

MI IQSERVER API

Version 3.0

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020

TABLE OF CONTENTS

REVISION HISTORY i

TABLE OF CONTENTS ii

1. 概述 1

1.1. 模块说明 1

1.2. 流程框图 1

1.3. 关键字说明 1

2. API 参考 2

2.1. MI_IQSERVER_Open 2

2.2. MI_IQSERVER_Close 3

2.3. MI_IQSERVER_SetDataPath 4

2.4. MI_IQSERVER_SetCustFunc 5

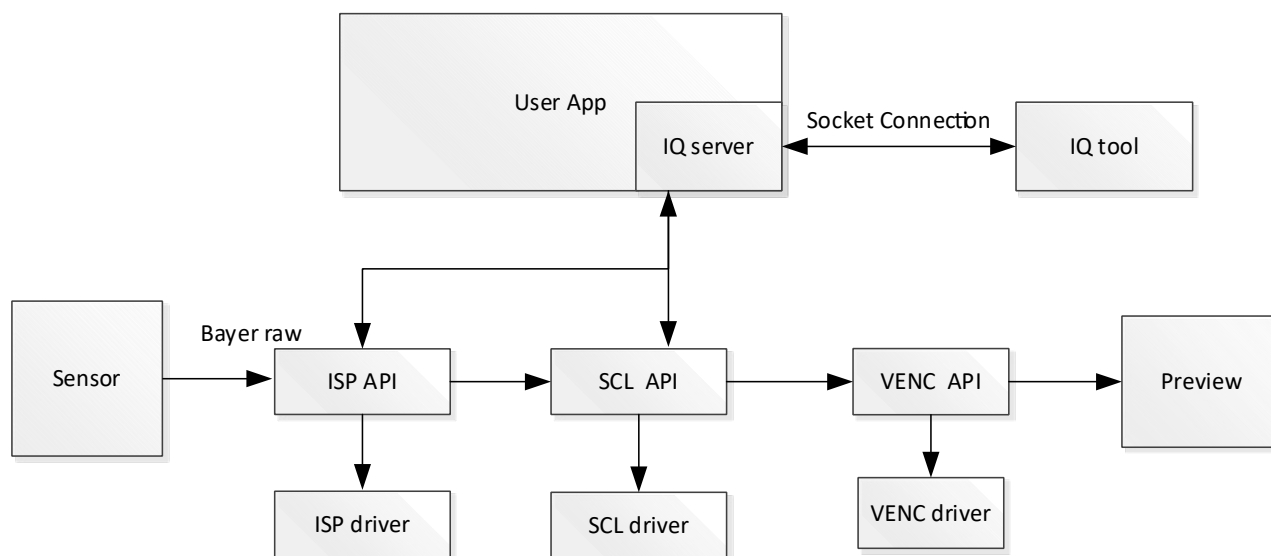
3. IQSERVER 错误码 7

1. 概述

1.1. 模块说明

IQSERVER(Image Quality tuning Server)图像质量调校服务，用来完成调校工具（IQ Tool）和开发板之间的数据通信，包括 ISP 参数设置，获取图像，上传/下载相关文件等功能。

1.2. 流程框图



1.3. 关键字说明

- ISP
Image Signal Processing 图像信号处理单元，负责图像降噪/颜色渲染/亮度调整等功能。
- SCL
Scale(缩放)。
- VENC
Video Encoder 视频编码器，负责将 YUV 序列编码成 H264/H265/Jpeg 等格式的视频流。

2. API 参考

该功能模块提供以下 API:

API 名	功能
MI_IQSERVER_Open	启动IQ server
MI_IQSERVER_Close	关闭IQ server
MI_IQSERVER_SetDataPath	设置IQ server数据存储路径
MI_IQSERVER_SetCustFunc	设置IQ server的可执行自定义函数

2.1. MI_IQSERVER_Open

➤ 功能

调用此接口启动 IQ server。

➤ 语法

```
MI_S32 MI_IQSERVER_Open();
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值 {
MI_IQSERVER_OK, 启动 IQ server 成功。
其它, 启动失败, 详情参照[错误码](#)。

➤ 依赖

- 头文件: `mi_iqserver.h`
- 库文件: `libmi_iqserver.a`

※ 注意

- IQ server目前只能创建一个。创建之后, 即可以用IQ Tool连接调试。

➤ 举例

```
/*declaration*/  
MI_S32 s32Ret = 0;
```

```

/* open IQSERVER*/
s32Ret = MI_IQSERVER_Open();
if (MI_IQSERVER_OK != s32Ret)
{
    Printf("open iqserver error:%x\n", s32Ret);
    return s32Ret;
}

```

➤ 相关主题

[MI_IQSERVER_Close](#)

2.2. MI_IQSERVER_Close

➤ 功能

调用此接口关闭 IQ server。

➤ 语法

```
MI_S32 MI_IQSERVER_Close();
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值 {

- MI_IQSERVER_OK, 成功关闭 IQ server。
- 其它, 关闭 IQ server 失败, 详情参照[错误码](#)。

➤ 依赖

- 头文件: `mi_iqserver.h`
- 库文件: `libmi_iqserver.a`

※ 注意

调用 [MI_IQSERVER_Open](#) 与 [MI_IQSERVER_Close](#) 需要对应。

➤ 举例

```

/*declaration*/
MI_S32 s32Ret = 0;
/* close IQSERVER*/
s32Ret = MI_IQSERVER_Close();
if (MI_IQSERVER_OK != s32Ret)
{
    Printf("close iqserver error:%x\n", s32Ret);
}

```

```
    return s32Ret;
}
```

➤ 相关主题

[MI_IQSERVER_Open](#)

2.3. MI_IQSERVER_SetDataPath

➤ 功能

设置IQ server的数据存储路径。

➤ 语法

```
MI_S32 MI_IQSERVER_SetDataPath(char* path);
```

➤ 形参

参数名称	描述	输入/输出
path	路径字符串指针，末尾是\0。	输入

➤ 返回值

返回值 {
MI_IQSERVER_OK, 成功。
其它, 设置失败, 详情参照[错误码](#)。

➤ 依赖

- 头文件: `mi_iqserver.h`
- 库文件: `libmi_iqserver.a`

➤ 注意

- 可以在MI_IQSERVER_Open之前调用。
- 调用前, 请确认路径是否可访问, 路径内是否有IQ server需要的数据资料。
- IQ server通常需要读取的资料有: `isp_api.xml` (用于IQ Tool使用)。

➤ 举例

```
/*declaration*/
MI_S32 s32Ret = 0;
char *data_path = "/customer/iqconfig/";
/* set IQSERVER data path*/
s32Ret = MI_IQSERVER_SetDataPath(data_path);
if (MI_IQSERVER_OK != s32Ret)
{
    Printf("set iqserver data path error:%x\n", s32Ret);
    return s32Ret;
}
```


- 相关主题
无。

2.4. MI_IQSERVER_SetCustFunc

- 功能

设置IQ server的可执行的自定义函数，满足用户通过IQ Server调用自定义接口函数的目的。

- 语法

```
MI_S32 MI_IQSERVER_SetCustFunc(MI_S32(* func)(MI_U16 data_type, MI_U32 length, MI_U8 * data));
```

- 形参

参数名称	描述	输入/输出
func	用户自定义的函数指针	输入

- 返回值

返回值 {
MI_IQSERVER_OK, 成功。
其它, 设置失败, 详情参照[错误码](#)。

- 依赖

- 头文件: mi_iqserver.h
- 库文件: libmi_iqserver.a

- ※ 注意

- 使用前，需要客户先实现自定义的func函数。函数参数如下：
 1. datatype: 表示 data 的类型，用户自定义，用来区分不同的 data。
 2. length: 表示 data 的长度，IQ Tool 会识别并自动填入该值。
 3. data: 表示 data 地址。
 客户在 func 内部可根据上述参数，做相关处理。
- 可以添加用户自定义的函数功能实现，IQ server内部获取IQ Tool传入的数据，再调用客户传进来的函数func，这样在func里面就可以获取到IQ Tool的数据，实现定制化的功能。
比如下面两项自定义功能：
 1. 通过IQ Tool传入固件，在func里面获取固件，然后烧写flash。
 2. 通过IQ Tool传入IR开关选项，在func里面切换IR。
- 调用前，请确认函数指针func是否可执行。
- Data_type和data数据通过IQ tool传入，length表示data的长度。
- Data_type可以用来区分data的类型，这样func可以针对不同的类型实现不同的功能。

- 举例

```
/*declaration*/  
MI_S32 s32Ret = 0;
```

```
MI_S32 sample_func(MI_U16 data_type, MI_U32 length, MI_U8 *data)
{
    printf("call MI_IQSERVER_SetCustFunc() success\n");
    printf("data_type is %d, length:%u, data addr:%p\n", data_type, length, data);
    //do something
    return MI_IQSERVER_OK;
}
/* set IQSERVER custom function*/
s32Ret = MI_IQSERVER_SetCustFunc(sample_func);
if (MI_IQSERVER_OK != s32Ret)
{
    Printf("set iqserver cust func error:%x\n", s32Ret);
    return s32Ret;
}
```

- 相关主题
无。

3. IQSERVER 错误码

IQSERVER API 错误码如下所示。

错误代码	宏定义	描述
0x0	MI_IQSERVER_OK	成功
0x1	MI_ERR_IQSERVER_FAIL	失败
0x2	MI_ERR_IQSERVER_NULL_PTR	空指针异常
0x3	MI_ERR_IQSERVER_NO_BUFF	Buff不够